GridGuard — Master Project Dossier

Autonomous Reliability Intelligence for Power Distribution Networks

Prepared for the AI in Production — Building & Scaling Real-World Intelligent Software Systems hackathon.

GridGuard — Project Proposal

One-line concept

GridGuard is an AI-powered reliability control plane that predicts transformer and feeder failure risk, explains the evidence, and simulates safer load-redistribution actions before an outage occurs.

Problem

Electricity networks are under pressure from increasing demand, distributed energy resources, EV charging, data centres, extreme weather, aging assets, and limited maintenance capacity. A utility may have thousands of transformers but only a small number of inspection crews. Reactive maintenance means teams discover problems after a customer-impacting failure.

The International Energy Agency identifies grid capacity as an emerging bottleneck and notes that annual grid investment must rise substantially by 2030. It also highlights technologies such as dynamic transformer rating and topology optimisation as ways to use existing assets more effectively. 1

GridGuard addresses the operational gap between raw telemetry and a safe, prioritized maintenance decision.

Product users

- **Grid operations engineers:** monitor assets and investigate abnormal behavior.
- **Maintenance planners:** prioritize inspections and replacement work.
- **Incident commanders:** understand the likely failure window and affected customers.
- **Reliability managers:** compare predicted risk with actual failures and model quality.

- **Integration teams:** consume stable APIs from dispatch, SMS, GIS, and work-order systems.

Product promise

GridGuard answers four questions:

1. **Which asset is likely to fail?**
2. **Why does the system believe that?**
3. **What can we do before failure?**
4. **Can we trust the prediction right now?**

Core workflow

1. Telemetry arrives from SCADA, smart meters, transformer sensors, weather feeds, and maintenance systems.
2. The ingestion gateway validates schemas, authenticates sources, deduplicates events, and records data quality.
3. The feature service creates rolling statistics, load-shape features, thermal stress indicators, and asset-age context.
4. The model router chooses among the primary risk model, anomaly detector, and deterministic fallback.
5. The API returns failure probability, time window, confidence, explanations, model version, and recommended action.
6. The counterfactual simulator estimates the effect of load redistribution or inspection priority changes.
7. Operators approve, reject, or modify the recommended action. Every decision is audited.

Differentiation

Most predictive-maintenance prototypes produce one risk score. GridGuard adds two production-grade ideas:

Risk-budgeted model routing

The system monitors not just model output but also sensor freshness, missingness, distribution drift, latency, and calibration. When the model cannot safely support a decision, GridGuard

switches to a conservative rules-based safety mode and clearly labels the result.

Counterfactual grid actions

GridGuard does not stop at “transformer TX-1042 is high risk.” It lets an operator evaluate a candidate action:

> Redistribute 15% of load to feeder F-22 for the next six hours.

It returns predicted risk reduction, overload risk created elsewhere, customers protected, energy-loss estimate, and confidence. The simulator is advisory and cannot issue switching commands.

Hackathon requirement mapping

Problem statement requirement	GridGuard demonstration
AI capability in a usable product	Asset failure prediction and anomaly detection in an operations dashboard
Well-designed, versioned APIs	`/v1/assets/{id}/risk`, `/v1/simulations/load-redistribution`, health and metrics endpoints
Cloud-native deployment	Stateless containerized services, Kubernetes Deployment, autoscaling, load balancing
CI/CD	Automated tests, image build, model evaluation gate, staging, canary, rollback
Monitoring and alerting	OpenTelemetry, Prometheus, Grafana, drift, latency, error, cost and fallback metrics
Latency/concurrency/cost trade-offs	Fast feature cache, asynchronous telemetry processing, model routing, request budgets
Security and versioning	OIDC, mTLS, RBAC, signed model artifacts, audit log, immutable model versions
Graceful failure	Sensor quality gate, deterministic thermal fallback, degraded confidence, operator alert

Success metrics

- Critical-failure recall: **≥ 0.95** on a held-out storm or overload test set.
- Calibration error: **≤ 0.05** for predicted failure probability.
- Prediction p95 latency: **<150 ms** for warm features.
- API availability: **99.9%** with two replicas and rolling deployment.
- Fallback response: **100%** when configured safety-gate conditions are violated.
- Complete audit coverage: **100%** of predictions contain model, route, evidence, and timestamp.
- Maintenance value: fewer emergency truck rolls and fewer unplanned outages in a simulated utility.

Boundaries

GridGuard is an operator-assistance system. It does not directly operate breakers, change protection settings, or authorize an outage. Any control action must pass through existing utility procedures and human approval.

References

GridGuard — Software Requirements Specification

1. Scope

GridGuard ingests asset telemetry and operational context, scores failure risk, detects anomalies, simulates safe interventions, and exposes the result through APIs and a web dashboard.

The first release targets distribution transformers and feeders. The design must support other asset classes later without changing the API versioning approach.

2. Functional requirements

FR-01 Asset registry

The system shall store a stable asset identifier, asset type, feeder, location, rated capacity, age, maintenance history, criticality class, and connected-customer estimate.

FR-02 Telemetry ingestion

The system shall accept timestamped voltage, current, load factor, winding temperature, ambient temperature, oil temperature where available, vibration, power factor, and sensor-health readings. It shall reject malformed payloads with a structured error.

FR-03 Data quality

The system shall calculate freshness, missingness, duplicate rate, range violations, and sensor-health status for every asset window.

FR-04 Risk prediction

The system shall return a failure probability, severity band, predicted window, confidence, model version, route, contributing factors, and recommended next action.

FR-05 Anomaly detection

The system shall identify unusual telemetry patterns even when no failure label is available and surface the anomaly type and evidence.

FR-06 Counterfactual simulation

The system shall accept a candidate load-redistribution scenario and return predicted risk change, downstream overload risk, customers protected, estimated energy loss, and confidence.

FR-07 Explainability

Every risk response shall include human-readable contributing factors and machine-readable feature contributions. Explanations shall never imply certainty beyond the confidence value.

FR-08 Model routing and fallback

The system shall route healthy requests to the primary model. If sensor health, data freshness, inference latency, or model confidence crosses a policy threshold, it shall use a deterministic safety fallback and identify that route in the response.

FR-09 Model lifecycle

Every model shall have a version, training-data snapshot, evaluation metrics, owner, approval status, and rollback target. A model shall not be promoted if it fails recall, calibration, latency, or fairness gates.

FR-10 Alerting

The system shall create alerts for critical asset risk, sensor failure, drift, elevated fallback rate, API errors, and SLO breach. Alerts shall have severity, acknowledgement, owner, and audit history.

FR-11 Dashboard

The dashboard shall show asset risk, feeder context, decision explanation, service health, model version, data quality, alert state, and simulation output.

FR-12 Auditability

The system shall record input hash, feature version, model version, route, output, explanation, operator action, and timestamps for every prediction.

3. Non-functional requirements

- **Availability:** 99.9% monthly for the online prediction API.
- **Latency:** p95 under 150 ms for warm online predictions; asynchronous ingestion may be eventually consistent.
- **Scalability:** scale horizontally to 500 requests/second in the demo benchmark and support partitioned telemetry ingestion.
- **Security:** OIDC authentication, role-based authorization, encryption in transit and at rest, secret rotation, and least privilege.
- **Resilience:** no single model-worker failure shall make the API unavailable; fallback must remain available.
- **Privacy:** avoid raw customer-level data when feeder aggregates are sufficient; apply retention and minimization policies.
- **Usability:** an operator must be able to understand why an asset is high-risk without opening a notebook.
- **Maintainability:** API schemas are backward compatible within a major version and have generated documentation.

4. Roles

Role	Permissions
Operator	View assets, acknowledge alerts, run simulations
Planner	Create work recommendations and export reports
Reliability manager	Approve model promotions and thresholds
Administrator	Manage identity, connectors, policies, and retention
Auditor	Read-only access to prediction and action history

5. Acceptance scenarios

1. Given a healthy transformer with stable telemetry, the system returns a normal or watch score within 150 ms.
2. Given a transformer with high temperature, rising load, voltage variance, and overdue maintenance, the system returns critical risk with at least two evidence factors.
3. Given stale or degraded sensors, the API returns a safety-fallback route and reduced confidence.
4. Given an unapproved model version, deployment promotion is blocked.
5. Given a load-redistribution scenario that overloads a neighbor feeder, the simulator rejects it and explains the constraint.
6. Given a critical alert, the dashboard shows acknowledgement state and audit history.

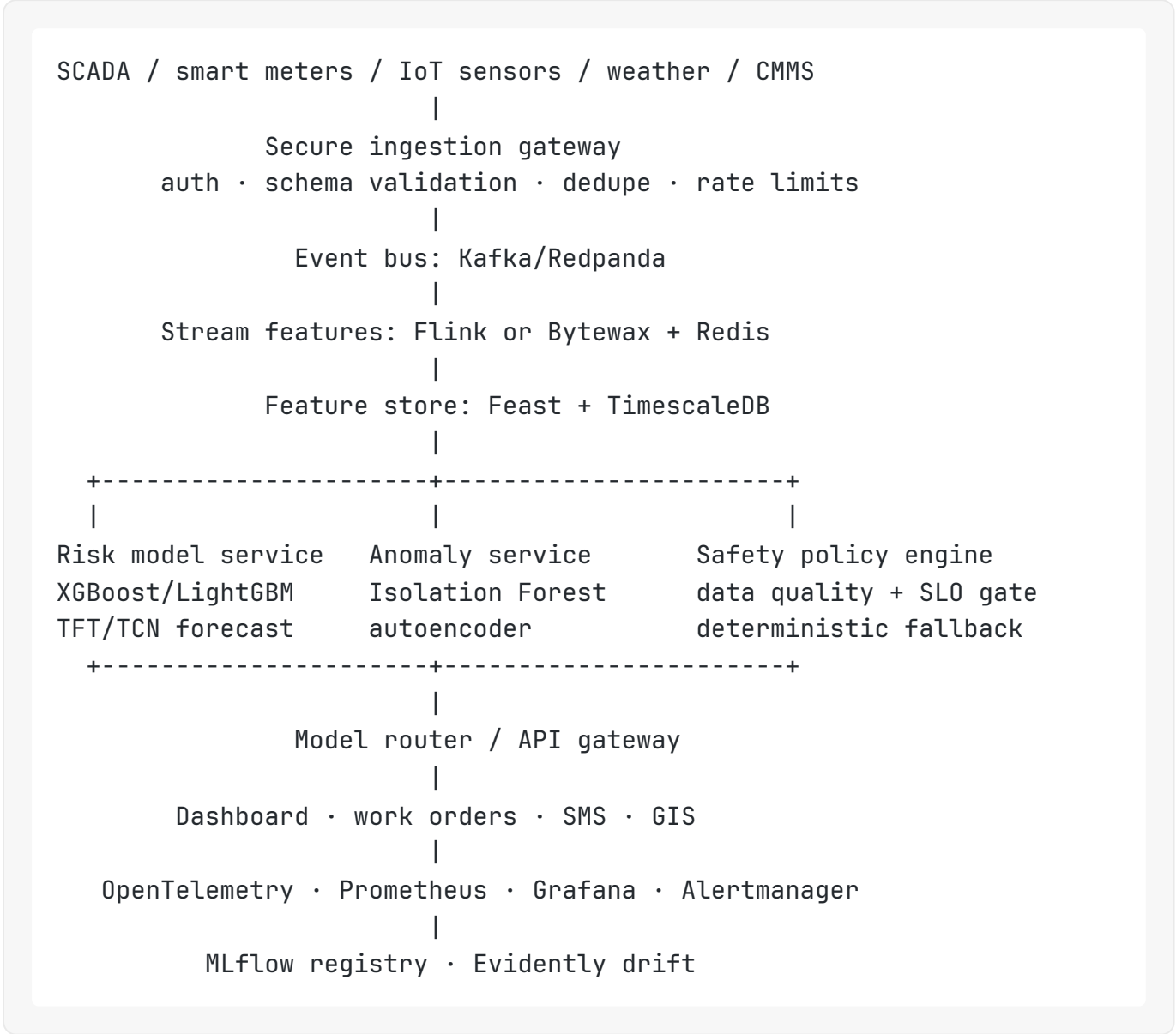
GridGuard — Architecture and MLOps Design

Architecture principle

Separate **telemetry**, **prediction**, **decision support**, and **control authority**. GridGuard may recommend an action, but existing utility control systems and human procedures retain authority

over physical switching.

Logical architecture



Recommended implementation stack

Layer	Technology	Reason
API	FastAPI + Pydantic	Typed, documented, low-latency APIs
Eventing	Redpanda or Kafka	Durable telemetry stream and replay
Online cache	Redis	Fast rolling features and rate limits
Time series	TimescaleDB	SQL analytics over telemetry windows
Geospatial	PostGIS + H3	Feeder, asset, and customer aggregation
Baseline model	XGBoost/LightGBM	Strong tabular performance and explainability
Forecast model	Temporal Fusion Transformer	Multi-horizon load and thermal forecasting
Anomaly model	Isolation Forest / autoencoder	Useful with sparse failure labels
Serving	KServe or BentoML	Versioning, canary, autoscaling
Registry	MLflow	Lineage, approval, rollback
Monitoring	OpenTelemetry + Prometheus + Grafana	Unified operational telemetry
ML monitoring	Evidently	Drift, quality, prediction distribution
Runtime	Docker + Kubernetes	Reproducible deployment and autoscaling
Identity	Keycloak or cloud OIDC	RBAC and service identity

Prediction flow

1. Authenticate request and validate schema.
2. Fetch latest feature vector from the online store.
3. Run data-quality and freshness checks.
4. Query the model router.
5. Calculate risk, interval, confidence, explanation, and route.
6. Write immutable prediction record.
7. Return the versioned response.

Model strategy

Baseline model

Use a gradient-boosted classifier with features such as:

- Rolling 5-minute, 1-hour, and 24-hour load percentiles
- Temperature rise rate and thermal headroom
- Voltage and current variance
- Power-factor trend
- Asset age and maintenance lateness
- Weather severity
- Historical outage count
- Feeder criticality and customer count

Advanced model

Add a Temporal Fusion Transformer or temporal convolutional network for multi-horizon load and temperature forecasts. Use it only after the baseline establishes a measurable improvement in recall and calibration.

Explainability

Use SHAP for tabular model contributions and counterfactual feature perturbation for the simulator. Explanations should be stored with the prediction rather than regenerated later.

Safe routing policy

Route to fallback when any of the following is true:

- Sensor health < 0.75
- Latest critical feature is older than the freshness threshold
- Inference p95 exceeds 900 ms
- Required features are missing
- Drift index exceeds the policy threshold
- Model confidence interval is too wide for the requested decision

Fallback output must include `route: safety-fallback`, lower confidence, the failed gate, and a conservative action such as inspection or load review.

Deployment topology

- Two or more API replicas behind a load balancer.
- Separate ingestion and online prediction deployments.
- Horizontal Pod Autoscaler on CPU, request rate, and queue depth.
- Canary model deployment with 5% shadow or live traffic.
- Pod disruption budget for API and router.
- Readiness probes for dependency health; liveness probes only for process health.
- Network policies limiting access between namespaces.

CI/CD and model promotion

```
Pull request
   → lint + unit tests + schema tests
   → build signed image
   → offline model evaluation
   → deploy staging
   → load and failure-injection tests
   → shadow traffic
   → canary 5%
   → reliability-manager approval
   → 100% promotion or automatic rollback
```

Promotion gates:

- Critical recall does not fall below the current champion.
- Calibration error is within the agreed bound.
- No unacceptable subgroup disparity.
- p95 latency and cost remain within budget.
- Fallback path passes failure-injection tests.

Failure modes

Failure	Detection	Response
Sensor feed stale	Freshness metric	Mark feature degraded; fallback
Model worker crash	Readiness probe	Route to replica/fallback
Feature store unavailable	Dependency health	Use last-known-safe features or fallback
Schema change	Contract test	Reject event and alert owner
Drift	Evidently monitor	Shadow retraining candidate; no auto-promotion
Traffic spike	Rate and queue metrics	Autoscale; protect critical tenants
Bad model release	Canary metrics	Automatic rollback

GridGuard — API Contract v1

Base path: `/api/v1`

Authentication: `Authorization: Bearer <OIDC token>`

All responses include `request_id` and timestamps. API contracts are designed for OpenAPI generation.

POST `/assets/{asset_id}/risk`

Request:

```
{
  "observations": {
    "load_factor": 0.91,
    "winding_temperature_c": 108.4,
    "ambient_temperature_c": 34.2,
    "voltage_variance": 0.08,
    "current_variance": 0.13,
    "power_factor": 0.82
  },
  "sensor_health": 0.97,
  "observed_at": "2026-10-13T01:10:00Z"
}
```

Response:

```
{
  "request_id": "req_01J9X",
  "asset_id": "TX-MUM-1042",
  "prediction": {
    "risk_score": 0.87,
    "severity": "critical",
    "failure_window_hours": {"low": 8, "high": 14},
    "confidence": 0.91,
    "route": "champion-model",
    "model_version": "gridguard-risk-0.4.1",
    "contributing_factors": [
      {"feature": "winding_temperature_c", "direction": "up", "impact": 0
      {"feature": "load_factor", "direction": "up", "impact": 0.24},
      {"feature": "maintenance_overdue_days", "direction": "up", "impact"
    ],
    "recommended_action": "Dispatch inspection crew and review feeder loa
  },
  "data_quality": {
    "freshness_seconds": 12,
    "missing_features": [],
    "status": "healthy"
  }
}
```

POST `/simulations/load-redistribution`

Request:

```
{
  "source_asset_id": "TX-MUM-1042",
  "target_feeder_id": "FDR-MUM-22",
  "redistribution_percent": 15,
  "duration_hours": 6
}
```

Response:

```
{
  "request_id": "sim_01J9Y",
  "scenario": {"redistribution_percent": 15, "duration_hours": 6},
  "result": {
    "status": "advisable_with_review",
    "source_failure_risk_before": 0.87,
    "source_failure_risk_after": 0.56,
    "target_overload_risk": 0.18,
    "customers_protected_estimate": 1840,
    "estimated_energy_loss_kwh": 42,
    "confidence": 0.78,
    "constraints": ["requires operator approval", "do not exceed target f
  }
}
```

GET `/assets?severity=critical&feeder_id=FDR-MUM-22`

Returns a paginated list of assets, risk state, last observation, model route, and acknowledgement status.

GET `/health`

Returns service and dependency status. This endpoint must not expose secrets.

GET `/metrics`

Returns or exports:

- request count and error count
- p50/p95/p99 latency
- queue depth
- fallback rate
- data freshness violations
- drift index
- model recall and calibration when labels are available
- cost estimate per 1,000 predictions

Error format

```
{
  "error": {
    "code": "FEATURES_STALE",
    "message": "Required telemetry is older than the configured freshness
    "retryable": false,
    "request_id": "req_01J9X"
  }
}
```

Compatibility policy

- Additive fields are allowed in a minor version.
- Removing or renaming fields requires a major version.
- Model versions are independent of API versions.
- Every model response must remain replayable from its recorded input and feature version.

GridGuard — Implementation Roadmap and Demo Script

Hackathon MVP scope

Build the smallest system that convincingly proves production behavior:

1. Synthetic telemetry generator for 50 transformers and 5 feeders.
2. Versioned risk API using a deterministic baseline model or trained gradient-boosted model.
3. Anomaly detector for temperature, voltage, and load spikes.
4. Safety fallback driven by sensor health and latency thresholds.
5. Dashboard with asset ranking, explanations, service metrics, and audit trace.
6. Counterfactual load-redistribution simulator using a simple network constraint model.
7. Dockerfile, CI tests, load test, and deployment manifest.

Suggested 48-hour plan

Phase 1 — Product contract

- Confirm asset schema, prediction response, thresholds, roles, and safety boundaries.
- Generate realistic normal, degraded, and failure-near telemetry.
- Define the champion baseline and evaluation metrics.

Phase 2 — AI and API

- Implement feature engineering and baseline risk model.
- Add anomaly detector and explanation output.
- Implement `/risk`, `/simulations/load-redistribution`, `/health`, and `/metrics`.
- Add contract and fallback tests.

Phase 3 — Operator experience

- Build the asset-risk table and feeder context view.

- Add a decision trace showing evidence, model route, confidence, and recommendation.
- Add simulation controls and compare before/after risk.

Phase 4 — Production proof

- Containerize services.
- Add CI pipeline and model evaluation gate.
- Run concurrent load test and inject model, sensor, and feature-store failures.
- Record p95 latency, fallback activation, and error recovery.

Phase 5 — Story and polish

- Create a two-minute pitch.
- Prepare architecture and failure-mode slides.
- Demonstrate that human approval remains required for operational changes.

Demo narrative

Scene 1 — The problem

“An operations team has thousands of assets but limited inspection capacity. Waiting for an outage is expensive and unsafe.”

Scene 2 — Normal operations

Open the dashboard. Show an asset list sorted by risk. Select `TX-MUM-1042`. Explain that high temperature, load factor, voltage variance, and overdue maintenance contribute to its score.

Scene 3 — AI prediction

Call the risk endpoint. Show the stable schema, model version, confidence, time window, and evidence factors.

Scene 4 — Counterfactual action

Simulate moving 15% of load to a neighboring feeder. Show that source failure risk decreases while target overload risk remains within the review threshold.

Scene 5 — Failure handling

Stop or delay the model worker, or set sensor health to 0.4. Show that the response changes to `safety-fallback`, confidence drops, and a data-quality alert appears.

Scene 6 — Scale and observability

Run concurrent requests. Show p95 latency, request count, fallback rate, drift index, and cost estimate. Explain autoscaling and rollback.

Scene 7 — Human safety boundary

State clearly: “GridGuard recommends and explains. It does not operate the grid. A qualified operator and existing utility controls own the final action.”

Evaluation matrix

Judge concern	Evidence to show
Architectural soundness	Architecture diagram and service boundaries
Reliability	Fallback injection, health endpoint, failure-mode table
Scalability	Concurrent load test and autoscaling design
Ease of integration	OpenAPI-compatible endpoints and stable schemas
Observability	Dashboard metrics and decision trace
Real-world readiness	Asset registry, maintenance workflow, audit log, role model
Novelty	Risk-budgeted routing and counterfactual load simulation

Stretch goals

- Use a real public outage or weather dataset for offline evaluation.
- Add graph-based feeder embeddings with a Graph Neural Network.
- Add ONNX edge inference for substations with intermittent connectivity.
- Add conformal prediction intervals and calibration plots.
- Integrate a work-order connector with a sandbox-only mock.

- Add fairness checks across neighborhoods and critical-customer classes.

GridGuard — Security, Safety, Ethics, and Risk Register

Safety position

GridGuard is decision support, not autonomous grid control. The system must never issue a breaker command, change protection settings, or claim that an outage is impossible. Recommendations must be reviewed by authorized personnel through existing procedures.

Security controls

- OIDC authentication for human users and service accounts.
- RBAC separating operators, planners, reliability managers, administrators, and auditors.
- mTLS between ingestion, feature, model, and API services.
- Signed container and model artifacts.
- Secrets stored outside source code and rotated regularly.
- Input validation, rate limiting, replay protection, and request IDs.
- Network policies restricting lateral movement.
- Immutable audit records for model and operator actions.
- Retention limits for customer and telemetry data.
- Dependency and image vulnerability scanning in CI.

AI safety controls

- Display confidence and uncertainty, not only a categorical label.
- Surface the model route and degraded state.
- Require evidence for high-severity recommendations.
- Block automatic promotion when recall or calibration degrades.
- Maintain a champion model and rollback target.

- Use human approval for load actions.
- Test adversarial and corrupted telemetry.
- Separate simulation from real control systems.

Responsible AI considerations

A maintenance-priority model can unintentionally under-serve neighborhoods with sparse sensors or incomplete historical records. GridGuard should therefore:

- report data coverage by feeder and neighborhood;
- avoid using protected characteristics or proxy features without governance;
- compare error rates across critical-customer and geographic groups;
- prioritize public-safety impact, not only financial return;
- keep an appeal and override workflow for qualified operators;
- document thresholds, trade-offs, and known blind spots.

Risk register

Risk	Impact	Mitigation
False negative before failure	High	Conservative thresholds, fallback, high recall gate, human review
False positive causes unnecessary dispatch	Medium	Calibrated probabilities, priority tiers, cost-aware evaluation
Stale sensor data	High	Freshness checks, quality score, fallback route
Model drift	High	Continuous monitoring, shadow retraining, no blind auto-promotion
Cyber compromise	High	OIDC, mTLS, signed artifacts, network policies, audit logs
Operator over-trust	High	Explanations, uncertainty, explicit advisory label, training
Sparse data in a neighborhood	Medium	Coverage metric, active sensor placement, uncertainty penalty
Unsafe simulated action	High	Hard network constraints and human approval; no control integration
Vendor lock-in	Medium	OpenAPI, containerized services, portable model artifacts
Excessive cloud cost	Medium	Budgets, autoscaling, batching, model router, cost metric

Incident response

1. Freeze the affected model or connector.
2. Route predictions to the known-safe fallback.
3. Preserve logs, model version, input hashes, and audit events.
4. Notify the reliability owner and security owner according to severity.

5. Compare predictions against ground truth and identify the blast radius.
6. Roll back or patch only after evaluation.
7. Record the incident and update tests, thresholds, or runbooks.